

Reviewer Pack — Noncommutative L^p Geometry of Clause Polarity Distributions...

Entry 7972d06696e0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 19:20:23 UTC

Noncommutative L^p Geometry of Clause Polarity Distributions Bounds Disjunctive Communication Complexity

Entry ID: 7972d06696e0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 19:20:23 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry (L^p Geometry) **Field B** (complexity object): Communication Complexity (Disjointness)

Statement:

{‘concrete’: “For any clause set C with n variables and m clauses, define the noncommutative L^p norm of the clause polarity distribution as $\|P(C)\|_p = (1/m)^{1/p} * \sum \{cl\} |polarity(cl)|^p$, where ‘polarity(cl)’ is the sum of signs of literals in clause ‘cl’. Then, for any constant $p \geq 2$ and $n \leq 40$, the randomized communication complexity of disjointness is lower-bounded by $\Omega(\|P(C)\|_p^{1/p})$ bits.”, ‘quantitative’: ‘ $E[CC_R(DISJ_n)] = \Theta(\|P(C_n)\|_p^{1/p})$ ’, ‘falsifier’: ‘Provide a clause set C_n with n variables and m clauses such that $\|P(C_n)\|_p < n^{-c}$ for any constant $c > 0$, but $CC_R(DISJ_n) \geq n^c$.’}

Rationale (proposer’s reasoning):

{‘explanation’: ‘The noncommutative L^p geometry provides a framework to study the distribution of clause polarities in a conjunctive normal form (CNF). By using this geometric structure, we can measure the complexity of determining disjointness, which is a fundamental problem in communication complexity. The conjecture suggests that a higher norm implies a higher communication complexity, potentially exposing a deep connection between noncommutative geometry and algorithmic complexity.’, ‘linkage’: ‘The nonstandard analysis of L^p norms can capture the distributional aspects of clause polarities, which are not directly captured by standard probability theory. This could reveal new insights into the structure of disjunctive problems.’}

Taxonomy category: COMM_DISJ (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e7caeca87f8f2c71

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

To support the conjecture, $\|P(C_n)\|_p^{(1/p)}$ must be less than or equal to $CC_R(DISJ_n)$ for all clause sets C_n with $n \leq 40$ and at least 80% of seeds, where $\|P(C_n)\|_p$ is the non-commutative L^p norm and $CC_R(DISJ_n)$ is the randomized communication complexity.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_clause_set(n, m):
        clauses = []
        for _ in range(m):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            clauses.append(clause)
        return clauses

    def lp_norm(p, clauses):
        n = len(clauses[0])
        total = 0
        for clause in clauses:
            polarity = sum(clause)
            total += abs(polarity) ** p
        return (1 / len(clauses)) ** (1 / p) * total

    def dpll_disjointness(n, m):
        # Simplified DPLL algorithm to check disjointness
        clauses = generate_clause_set(n, m)
        for i in range(2**n):
            assignment = [((i >> j) & 1) * 2 - 1 for j in range(n)]
            if all(any(lit * assign != 0 for lit in clause) for clause in clauses):
                return True
```

```

        return False

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    m = random.randint(1, n * (n + 1) // 2)
    clauses = generate_clause_set(n, m)
    lp_norm_value = lp_norm(2, clauses)
    cc_disjointness = dpll_disjointness(n, m)

    results.append({
        "metric_name": "communication_complexity",
        "metric_value": cc_disjointness,
        "instances_tested": 1,
        "conjecture_holds": lp_norm_value <= 1 if cc_disjointness else False,
        "counterexample": f"n={n}, m={m}, lp_norm={lp_norm_value}, cc_disjointness={cc_disjointness}"
    })

return {
    "seed": seed,
    **results[0],
    "mean_metric_value": sum(result["metric_value"] for result in results) / len(results),
    "support_fraction": sum(1 for result in results if result["conjecture_holds"]) / len(results)
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["mean_metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["support_fraction"] >= 0.8) / len(results)

    if all(result["support_fraction"] >= 0.8 for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='first failing seed' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

```

Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cd0a81c3.py", line 76, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cd0a81c3.py", line 52, in run_trial
    cc_disjointness = dpll_disjointness(n, m)
                      ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cd0a81c3.py", line 41, in dpll_disjoint
    if all(any(lit * assign != 0 for lit in clause) for clause in clauses):
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cd0a81c3.py", line 41, in <genexpr>
    if all(any(lit * assign != 0 for lit in clause) for clause in clauses):
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cd0a81c3.py", line 41, in <genexpr>
    if all(any(lit * assign != 0 for lit in clause) for clause in clauses):
              ^^^^^
NameError: name 'assign' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means we cannot verify the conjecture's support or falsification. | next: Re-run the test with proper error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14286
2	preregistration	ollama_remote	glm4:latest	0	0	6075
3	novelty	ollama_remote	glm4:latest	0	0	4833
4	novelty	ollama_remote	glm4:latest	0	0	5141
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15912
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8777
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11054
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9477
9	judge	ollama_remote	glm4:latest	0	0	11107

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 86662 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7972d06696e0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7972d06696e0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7972d06696e0.tar.gz` (if generated)